10.5 Sets, Multisets, and Multimaps

We conclude this chapter by examining several additional abstractions that are closely related to the map ADT, and that can be implemented using data structures similar to those for a map.

- A ***set*** is an unordered collection of elements, without duplicates, that typically supports efficient membership tests. In essence, elements of a set are like keys of a map, but without any auxiliary values.
- A ***multiset*** (also known as a ***bag***) is a set-like container that allows duplicates.
- A ***multimap*** is similar to a traditional map, in that it associates values with keys; however, in a multimap the same key can be mapped to multiple values. For example, the index of this book maps a given term to one or more locations at which the term occurs elsewhere in the book.

10.5.1 The Set ADT

Python provides support for representing the mathematical notion of a set through the built-in classes **frozenset** and **set**, as originally discussed in Chapter 1, with frozenset being an immutable form. Both of those classes are implemented using hash tables in Python.

Python's collections module defines abstract base classes that essentially mirror these built-in classes. Although the choice of names is counterintuitive, the abstract base class collections.Set matches the concrete frozenset class, while the abstract base class collections.MutableSet is akin to the concrete set class.

In our own discussion, we equate the "set ADT" with the behavior of the built-in set class (and thus, the collections.MutableSet base class). We begin by listing what we consider to be the five most fundamental behaviors for a set S:

S.add(e): Add element e to the set. This has no effect if the set already contains e.

S.discard(e): Remove element e from the set, if present. This has no effect if the set does not contain e.

e in S: Return True if the set contains element e. In Python, this is implemented with the special __contains__ method.

len(S): Return the number of elements in set S. In Python, this is implemented with the special method __len__.

iter(S): Generate an iteration of all elements of the set. In Python, this is implemented with the special method __iter__.

In the next section, we will see that the above five methods suffice for deriving all other behaviors of a set. Those remaining behaviors can be naturally grouped as follows. We begin by describing the following additional operations for removing one or more elements from a set:

S.remove(e): Remove element e from the set. If the set does not contain e, raise a KeyError.

S.pop( ): Remove and return an arbitrary element from the set. If the set is empty, raise a KeyError.

S.clear( ): Remove all elements from the set.

The next group of behaviors perform Boolean comparisons between two sets.

S == T: Return True if sets S and T have identical contents.

S != T: Return True if sets S and T are not equivalent.

S <= T: Return True if set S is a subset of set T.

S < T: Return True if set S is a *proper* subset of set T.

S >= T: Return True if set S is a superset of set T.

S > T: Return True if set S is a *proper* superset of set T.

S.isdisjoint(T): Return True if sets S and T have no common elements.

Finally, there exists a variety of behaviors that either update an existing set, or compute a new set instance, based on classical set theory operations.

S | T: Return a new set representing the union of sets S and T.

S |= T: Update set S to be the union of S and set T.

S & T: Return a new set representing the intersection of sets S and T.

S &= T: Update set S to be the intersection of S and set T.

S ^ T: Return a new set representing the symmetric difference of sets S and T, that is, a set of elements that are in precisely one of S or T.

S ^= T: Update set S to become the symmetric difference of itself and set T.

S − T: Return a new set containing elements in S but not T.

S −= T: Update set S to remove all common elements with set T.

10.5.2 Python's MutableSet Abstract Base Class

To aid in the creation of user-defined set classes, Python's collections module provides a MutableSet abstract base class (just as it provides the MutableMapping abstract base class discussed in Section 10.1.3). The MutableSet base class provides concrete implementations for all methods described in Section 10.5.1, except for five core behaviors (add, discard, __contains__, __len__, and __iter__) that must be implemented by any concrete subclass. This design is an example of what is known as the ***template method pattern***, as the concrete methods of the MutableSet class rely on the presumed abstract methods that will subsequently be provided by a subclass.

For the purpose of illustration, we examine algorithms for implementing several of the derived methods of the MutableSet base class. For example, to determine if one set is a proper subset of another, we must verify two conditions: a proper subset must have size strictly smaller than that of its superset, and each element of a subset must be contained in the superset. An implementation of the corresponding __lt__ method based on this logic is given in Code Fragment 10.14.

```
def __lt__(self, other):       # supports syntax S < T
  """Return true if this set is a proper subset of other."""
  if len(self) >= len(other):
    return False               # proper subset must have strictly smaller size
  for e in self:
    if e not in other:
      return False             # not a subset since element missing from other
  return True                  # success; all conditions are met
```

Code Fragment 10.14: A possible implementation of the MutableSet.__lt__ method, which tests if one set is a proper subset of another.

As another example, we consider the computation of the union of two sets. The set ADT includes two forms for computing a union. The syntax S | T should produce a new set that has contents equal to the union of existing sets S and T. This operation is implemented through the special method __or__ in Python. Another syntax, S |= T is used to *update* existing set S to become the union of itself and set T. Therefore, all elements of T that are not already contained in S should be added to S. We note that this "in-place" operation may be implemented more efficiently than if we were to rely on the first form, using the syntax S = S | T, in which identifier S is reassigned to a new set instance that represents the union. For convenience, Python's built-in set class supports named version of these behaviors, with S.union(T) equivalent to S | T, and S.update(T) equivalent to S |= T (yet, those named versions are not formally provided by the MutableSet abstract base class).

```
def __or__(self, other):                    # supports syntax S | T
  """Return a new set that is the union of two existing sets."""
  result = type(self)( )                    # create new instance of concrete class
  for e in self:
    result.add(e)
  for e in other:
    result.add(e)
  return result
```

Code Fragment 10.15: An implementation of the MutableSet.__or__ method, which computes the union of two existing sets.

An implementation of the behavior that computes a new set as a union of two others is given in the form of the __or__ special method, in Code Fragment 10.15. An important subtlety in this implementation is the instantiation of the resulting set. Since the MutableSet class is designed as an abstract base class, instances must belong to a concrete subclass. When computing the union of two such concrete instances, the result should presumably be an instance of the same class as the operands. The function type(self) returns a reference to the actual class of the instance identified as self, and the subsequent parentheses in expression type(self)( ) call the default constructor for that class.

In terms of efficiency, we analyze such set operations while letting n denote the size of S and m denote the size of set T for an operation such as S | T. If the concrete sets are implemented with hashing, the expected running time of the implementation in Code Fragment 10.15 is $O(m+n)$, because it loops over both sets, performing constant-time operations in the form of a containment check and a possible insertion into the result.

Our implementation of the in-place version of a union is given in Code Fragment 10.16, in the form of the __ior__ special method that supports syntax S |= T. Notice that in this case, we do not create a new set instance, instead we modify and return the existing set, after updating its contents to reflect the union operation. The in-place version of the union has expected running time $O(m)$ where m is the size of the second set, because we only have to loop through that second set.

```
def __ior__(self, other):                   # supports syntax S |= T
  """Modify this set to be the union of itself an another set."""
  for e in other:
    self.add(e)
  return self                               # technical requirement of in-place operator
```

Code Fragment 10.16: An implementation of the MutableSet.__ior__ method, which performs an in-place union of one set with another.

10.5.3 Implementing Sets, Multisets, and Multimaps

Sets

Although sets and maps have very different public interfaces, they are really quite similar. A set is simply a map in which keys do not have associated values. Any data structure used to implement a map can be modified to implement the set ADT with similar performance guarantees. We could trivially adapt any map class by storing set elements as keys, and using None as an irrelevant value, but such an implementation is unnecessarily wasteful. An efficient set implementation should abandon the _Item composite that we use in our MapBase class and instead store set elements directly in a data structure.

Multisets

The same element may occur several times in a multiset. All of the data structures we have seen can be reimplemented to allow for duplicates to appear as separate elements. However, another way to implement a multiset is by using a *map* in which the map key is a (distinct) element of the multiset, and the associated value is a count of the number of occurrences of that element within the multiset. In fact, that is essentially what we did in Section 10.1.2 when computing the frequency of words within a document.

Python's standard collections module includes a definition for a class named Counter that is in essence a multiset. Formally, the Counter class is a subclass of dict, with the expectation that values are integers, and with additional functionality like a most_common(n) method that returns a list of the n most common elements. The standard __iter__ reports each element only once (since those are formally the keys of the dictionary). There is another method named elements( ) that iterates through the multiset with each element being repeated according to its count.

Multimaps

Although there is no multimap in Python's standard libraries, a common implementation approach is to use a standard map in which the value associated with a key is itself a container class storing any number of associated values. We give an example of such a MultiMap class in Code Fragment 10.17. Our implementation uses the standard dict class as the map, and a list of values as a composite value in the dictionary. We have designed the class so that a different map implementation can easily be substituted by overriding the class-level _MapType attribute at line 3.

```
class MultiMap:
  """A multimap class built upon use of an underlying map for storage."""
  _MapType = dict                    # Map type; can be redefined by subclass

  def __init__(self):
    """Create a new empty multimap instance."""
    self._map = self._MapType( )          # create map instance for storage
    self._n = 0

  def __iter__(self):
    """Iterate through all (k,v) pairs in multimap."""
    for k,secondary in self._map.items( ):
      for v in secondary:
        yield (k,v)

  def add(self, k, v):
    """Add pair (k,v) to multimap."""
    container = self._map.setdefault(k, [ ])      # create empty list, if needed
    container.append(v)
    self._n += 1

  def pop(self, k):
    """Remove and return arbitrary (k,v) with key k (or raise KeyError)."""
    secondary = self._map[k]                      # may raise KeyError
    v = secondary.pop( )
    if len(secondary) == 0:
      del self._map[k]                            # no pairs left
    self._n -= 1
    return (k, v)

  def find(self, k):
    """Return arbitrary (k,v) pair with given key (or raise KeyError)."""
    secondary = self._map[k]                      # may raise KeyError
    return (k, secondary[0])

  def find_all(self, k):
    """Generate iteration of all (k,v) pairs with given key."""
    secondary = self._map.get(k, [ ])             # empty list, by default
    for v in secondary:
      yield (k,v)
```

Code Fragment 10.17: An implementation of a MultiMap using a dict for storage. The __len__ method, which returns self._n, is omitted from this listing.